

SymPy Cheat Sheet

companion to *Symbolic Computation with SymPy* • `import sympy as sp`

Import

import SymPy; every command below starts with `sp.`

```
import sympy as sp
```

check the installed version

```
sp.__version__
```

Capital or lowercase?

Capitalised names are *classes*: they *create an object* that SymPy stores and works with

```
sp.Symbol, sp.Matrix, sp.Rational, sp.Function
```

classes that build an *unevaluated* object: written, but not computed (use `.doit()`)

```
sp.Integral, sp.Derivative, sp.Eq, sp.Piecewise
```

lowercase names are *functions*: they *do* something and return the result

```
sp.symbols, sp.diff, sp.integrate, sp.solve
```

pairs: one object vs. the helper that makes several

```
sp.Symbol('x') vs sp.symbols('x y')
```

pairs: unevaluated $\int f dx$ vs. the computed antiderivative

```
sp.Integral(f, x) vs sp.integrate(f, x)
```

mathematical functions stay lowercase, as in mathematics

```
sp.sqrt, sp.exp, sp.sin, sp.log
```

constants: *e* and *i* are capital E, I so they do not clash with your own variables `e`, `i`

```
sp.pi, sp.oo, sp.E, sp.I
```

Symbols

one symbol

```
x = sp.Symbol('x')
```

several symbols

```
x, y, z = sp.symbols('x y z')
```

numbered symbols ($x_1, \dots, x_5$); `list(X)` for a list

```
X = sp.symbols('x1:6')
```

$x_1, \dots, x_n$ for a Python integer n

```
sp.symbols(f'x1:{n+1}')
```

range of letters (a, b, c, d)

```
sp.symbols('a:d')
```

assumptions: also **real**, **integer**, **nonnegative**

```
sp.Symbol('t', positive=True)
```

one undefined function; use it as $f(x)$

```
f = sp.Function('f')
```

several undefined functions at once: `cls` (class) makes **symbols** create **Function** objects instead of **Symbols**

```
f, g = sp.symbols('f g', cls=sp.Function)
```

numbered undefined functions (u_1, u_2, u_3); call them as `u[0](x)`

```
u = sp.symbols('u1:4', cls=sp.Function)
```

Exact numbers and constants

exact $\frac{1}{3}$ (but $1/3$ is the float $0.333\dots$)

```
sp.Rational(1, 3)
```

exact square root, simplified: $\sqrt{8} = 2\sqrt{2}$

```
sp.sqrt(8)
```

constants π , e , i , ∞

```
sp.pi, sp.E, sp.I, sp.oo
```

symbolic functions (`np.exp` only for arrays)

```
sp.exp(x), sp.log(x), sp.sin(x)
```

absolute value $|x|$

```
sp.Abs(x)
```

Rewriting expressions

multiply out: $(x+1)^2 \rightarrow x^2 + 2x + 1$

```
sp.expand((x + 1)**2)
```

factor: $x^2 - 1 \rightarrow (x-1)(x+1)$

```
sp.factor(x**2 - 1)
```

try many rules; $\sin^2 x + \cos^2 x \rightarrow 1$

```
sp.simplify(e)
```

cancel common factors of a fraction

```
sp.cancel(e)
```

partial fractions

```
sp.apart(e, x)
```

group by powers of x

```
sp.collect(e, x)
```

substitute values or expressions

```
e.subs(x, 2)  
e.subs({x: 2, y: 3})
```

test whether two expressions are equal

```
sp.simplify(e1 - e2) == 0
```

stays unless x is **real** ($\rightarrow |x|$) or **positive** ($\rightarrow x$)

```
sp.sqrt(x**2)
```

Printing

plain text, e.g. `x**2 + 1`

```
print(e)
```

ASCII art

```
sp.pprint(e)
```

last line of each cell rendered as math

```
sp.init_printing()
```

render from anywhere in a cell

```
display(e)
```

LaTeX code as a string, e.g. $x^2 + 1$

```
sp.latex(e)
```

LaTeX string wrapped in $\dots$

```
sp.latex(e, mode='inline')
```

LaTeX inside a numbered **equation** environment; `'equation*'` for no number

```
sp.latex(e, mode='equation')
```

an equation `lhs = rhs`

```
sp.latex(sp.Eq(lhs, rhs))
```

matrix with round brackets

```
sp.latex(M, mat_str='pmatrix', mat_delim='')
```

mix your own LaTeX with SymPy's

```
display(Math(r'f(x) = ' + sp.latex(e)))
```

internal expression tree

```
sp.srepr(e)
```

Functions

expression; evaluate with `f_expr.subs(x, 3)`

```
f_expr = x**2 * sp.exp(-x)
```

Python function returning an expression

```
def f(t):  
    return t**2 * sp.exp(-t)
```

SymPy function object; `f(3)`

```
f = sp.Lambda(x, x**2 * sp.exp(-x))
```

rules for undefined functions (product rule)

```
sp.diff(f(x) * g(x), x)
```

solve the ODE $f'' + f = 0$

```
sp.dsolve(f(x).diff(x, 2) + f(x), f(x))
```

$$\begin{cases} -x & x < 0 \\ x^2 & 0 \leq x < 1; \text{first true condition wins} \\ x & \text{otherwise} \end{cases}$$

```
sp.Piecewise((-x, x < 0),  
(x**2, sp.And(0 <= x, x < 1)),  
(x, True))
```

condition *and* (both true)

```
sp.And(a, b) (a) & (b)
```

condition *or* (at least one true)

```
sp.Or(a, b) (a) | (b)
```

condition *not*

```
sp.Not(a) ~(a)
```

chained comparisons and Python's **and/or do not** work on symbols (**TypeError**)

```
0 <= x < 1, and, or
```

Solving equations

list of exact solutions of $e = 0$

```
sp.solve(e, x)
```

solve `lhs = rhs`

<code>sp.solve(sp.Eq(lhs, rhs), x)</code>
solve a system; returns a dict { x : ..., y : ...}
<code>sp.solve([e1, e2], [x, y])</code>
all real solutions, as a set (may be infinite or empty)
<code>sp.solveset(e, x, domain=sp.S.Reals)</code>
roots of a polynomial with multiplicities, { root : multiplicity}
<code>sp.roots(p, x)</code>
real roots of a polynomial; <code>.evalf()</code> for decimals
<code>sp.real_roots(p)</code>
numeric root near the initial guess 1
<code>sp.nsolve(sp.cos(x) - x, x, 1)</code>

Derivatives and series

first derivative $\frac{df}{dx}$
<code>sp.diff(f, x)</code>
second derivative $\frac{d^2 f}{dx^2}$
<code>sp.diff(f, x, 2)</code>
mixed partial $\frac{\partial^2 F}{\partial x \partial y}$
<code>sp.diff(F, x, y)</code>

partial, twice in x and once in y : $\frac{\partial^3 F}{\partial x^2 \partial y}$

<code>sp.diff(F, x, 2, y)</code>
unevaluated $\frac{d}{dx}f$; <code>.doit()</code> evaluates
<code>sp.Derivative(f, x)</code>
gradient ∇F
<code>[sp.diff(F, x), sp.diff(F, y)]</code>
Hessian matrix
<code>sp.hessian(F, (x, y))</code>
Taylor series to order n ; <code>.remove0()</code>
<code>sp.series(f, x, x0, n)</code>
tangent line at x_0
<code>f.subs(x, x0) + sp.diff(f, x).subs(x, x0)*(x - x0)</code>

Limits

right limit $\lim_{x \rightarrow x_0^+} f$ (default `dir='+'`)

<code>sp.limit(f, x, x0)</code>
left limit $\lim_{x \rightarrow x_0^-} f$
<code>sp.limit(f, x, x0, dir='-')</code>
two-sided limit; error if left $\neq$ right
<code>sp.limit(f, x, x0, dir='+-')</code>
limit at infinity $\lim_{x \rightarrow \infty} f$
<code>sp.limit(f, x, sp.oo)</code>

Piecewise at a break point: `sp.limit` is unreliable, so take the limit of the piece on each side

`sp.limit(left.piece, x, x0, dir='-')`
`sp.limit(right.piece, x, x0, dir='+')`

Integrals

indefinite integral $\int f \, dx$ (no $+C$)

<code>sp.integrate(f, x)</code>
definite integral $\int_a^b f \, dx$
<code>sp.integrate(f, (x, a, b))</code>
improper integral $\int_0^\infty f \, dx$
<code>sp.integrate(f, (x, 0, sp.oo))</code>

double integral $\int_0^1 \int_0^x f \, dy \, dx$ (inner first)

<code>sp.integrate(f, (y, 0, x), (x, 0, 1))</code>
unevaluated; <code>.doit()</code> or <code>.evalf()</code>
<code>sp.Integral(f, (x, a, b))</code>

check an antiderivative: should be 0

`sp.simplify(sp.diff(F, x) - f)`

Evaluation and NumPy

decimal with 20 digits

`e.evalf(20)`, `sp.N(e, 20)`

substitute and evaluate

`e.evalf(subs={x: 2})`

Python float (no symbols left)

<code>float(e)</code>
fast NumPy function: <code>f_np(np.linspace(0, 1, 100))</code>
<code>f_np = sp.lambdify(x, e, 'numpy')</code>

several variables

`sp.lambdify((x, y), e, 'numpy')`

list of SymPy numbers $\rightarrow$ NumPy array

`np.array(sp_list, dtype=float)`

SymPy matrix $\rightarrow$ NumPy array

`np.array(M, dtype=float)`

constant expression: `lambdify` returns a scalar, not an array

`np.full_like(xs, c)`

Linear algebra

matrix; `sp.eye(n)`, `sp.zeros(m, n)`

`A = sp.Matrix([[1, 2], [3, 4]])`

transpose A^T , inverse A^{-1}

`A.T`, `A.inv()`

matrix product, power, scalar multiple

`A @ B`, `A**3`, `2 * A`

elementwise product $A \odot B$

`A.multiply_elementwise(B)`

apply a function to every entry

`A.applyfunc(lambda e: e**2)`

determinant $\det A$, rank

`A.det()`, `A.rank()`

reduced row echelon form and pivot columns

`R, pivots = A.rref()`

bases (lists of column vectors)

`A.nullspace()`, `A.columnspace()`

eigenvalues as {**eigenvalue**: algebraic multiplicity}

`A.eigenvals()`

eigenvectors as [(**eigenvalue**, mult., [basis vectors]), ...]

`A.eigenvects()`

eigenspace of λ

`(A - lam * sp.eye(n)).nullspace()`

diagonalize $A = PDP^{-1}$ (test first with `A.is_diagonalizable()`)

`P, D = A.diagonalize()`

power A^n for a symbolic integer n

`P @ D**n @ P.inv()`

Plotting recipe

compute symbolically, `lambdify`, `plot`, label with `sp.latex`

```
f_np = sp.lambdify(x, f_expr, 'numpy')
xs = np.linspace(0, 8, 400)
fig, ax = plt.subplots()
ax.plot(xs, f_np(xs),
        label=f'${sp.latex(f_expr)}$')
ax.legend(); plt.show()
```

shade where the boolean mask `m` is true

`ax.fill_between(xs[m], f_np(xs[m]), alpha=0.3)`

quick look without NumPy

`sp.plot(f_expr, (x, 0, 8))`

Python helpers

build a list with a loop

```
out = []
for v in vals:
    out.append(v**2)
```

same, as a list comprehension

```
out = [v**2 for v in vals]
```

loop $i = 0, 1, \dots, N$

```
for i in np.arange(0, N + 1):
```

LaTeX output

raw string: backslashes are kept

`r'\frac{a}{b}'`

f-string: `\\` for `\`, `{{ }}` for literal braces

`f'${"\int_{{0}}^{{1}} {sp.latex(e)}\\,dx$}'`

Matplotlib math: no environments

`ax.set_title(r'${"\int_0^1 x\\,dx$}')`